AI Agent for Academic Guidance

Project Report for Milestone 3 of Software Engineering

Submitted by

Team Number: Team 6

Course Term: January 2025

IIT Madras BS Degree Program

Indian Institute of Technology Madras, Chennai

Tamil Nadu, India, 600036

Table of Contents

1. Project Schedule and Tools

1.1 Complete Schedule

The whole project is divided into different Software Development Lifecycle (SDLC) phases consisting of Planning, Designing, Development, Testing, and Maintenance. We have aligned these phases with the Milestones and Submission Cycles to ensure timely delivery.

The following is the complete schedule of development and submission for the project:

SDLC Phase	Milestones	Tasks	Sprint	Assignee	Start Date	End Date
Planning	Cycle 1 (Requirement Gathering & Analysis)	Identify primary, secondary, and tertiary users	1	Shruti/Himanshu/Puja	7-Jan	10-Jan
		List the required new features & integrations		Satyaki/Adarsh/Aaruni	11-Jan	13-Jan
		Define functional & non-functional requirements		Shreya/Puja/Shruti	14-Jan	15-Jan
		Write User Stories in standard format		Shruti/Himanshu/Adarsh	16-Jan	18-Jan
		Review & finalize project scope		Satyaki/Shreya	19-Jan	21-Jan
Analysis	Cycle 2 (UI/UX Design & Wireframing)	Develop Storyboard showing user interactions	2	Satyaki/Himanshu	22-Jan	24-Jan
		Create Low-Fidelity Wireframes		Shreya/Puja	25-Jan	27-Jan
		Define Navigation Flow & UX Guidelines		Shruti/Adarsh	28-Jan	31-Jan
		Apply Usability Heuristics		Aaruni/Adarsh/Himanshu	1-Feb	3-Feb
		Conduct UX Review & Revisions		Shreya/Himanshu/puja	4-Feb	5-Feb
Design	Cycle 3 (Scheduling & Software Design)	Define Sprints & Iterations	3	Shruti/Himanshu	6-Feb	7-Feb
		Set up Scrum Meeting Schedule		Shruti/Puja	8-Feb	9-Feb
		Identify components & modules		Shreya/Satyaki	10-Feb	13-Feb
		Develop Class & Flow Diagrams		Shruti/Adarsh	14-Feb	15-Feb
		Ensure page connectivity & navigation		Aaruni/Puja	16_Feb	20-Feb
Development	Cycle 4 (API Development & Backend Integration)	Define API Endpoints	4	Himanshu/Adarsh/Puja	21-Feb	23-Feb
		Develop & Integrate APIs		Satyaki/Shruti/Aaruni	24-Feb	28-Feb
		Implement CRUD Operations		Shruti/Adarsh/Shreya	1-Mar	3-Mar
		Document APIs using Swagger/OpenAPI		Adarsh/Shreya/Aaruni	4-Mar	5-Mar
		Ensure API Security & Authentication		Satyaki/Shruti/Aaruni	6-Mar	7-Mar
Testing	Cycle 5 (Test Cases & QA Process)	Design Test Cases for APIs	5	Shruti/Satyaki/Himanshu	8-Mar	12-Mar
		Perform Unit Testing		Shruti/Satyaki/Himanshu	13-Mar	16-Mar
		Conduct Integration Testing		Adarsh/Aaruni	14-Mar	18-Mar
		Implement Automated Tests		Adarsh/Aaruni	19-Mar	21-Mar
		Document Test Execution Results		Shreya/Puja	22-Mar	23-Mar
Deployment	Cycle 6 (Final Submission & Deployment)	Finalize Codebase & Review	6	Satyaki/Shruti/Shreya	23-Mar	24-Mar
		Deploy Working Prototype		Aaruni/Adarsh/Himanshu	25-Mar	26-Mar
		Prepare Final Report		Puja/Adarsh/Shruti	27-Mar	28-Mar
		Prepare Presentation & Demo Video		Shreya/Satyaki/Himanshu	28-Mar	29-Mar
		Submit Documentation & Setup Guide		Shreya	29-Mar	30-Mar

1.2 Sprint Schedule

We are using JIRA software to plan our sprints on the SCRUM board while following the Agile methodology. The project is divided into sprints of 1 to 3 weeks, each aligned with milestone submission deadlines. This alignment ensures that all tasks are completed within their respective submission dates.

The detailed sprint schedule is as follows:

A. Sprint 1: Milestone 1 & 2 - User Requirements and UI Design

Dates: 07/Jan/25 - 21/Jan/25 (2 weeks)

Goals:

- Identify users and create user stories for Milestone 1 Submission
- Develop storyboards and low-fidelity wireframes for Milestone 2 Submission
- Define UX guidelines and navigation flow
- Document functional & non-functional requirements

B. Sprint 2: Milestone 3 - Scheduling & UI Development

Dates: 22/Jan/25 - 05/Feb/25 (2 weeks)

Goals:

- Create a software development schedule
- Develop UI components and design Class Diagrams
- Create SCRUM documentation and Jira workflow
- Start initial frontend implementation (React/Vue.js)

C. Sprint 3: Milestone 4 - API Design and Implementation

Dates: 06/Feb/25 - 20/Feb/25 (2 weeks)

Goals:

- Design and implement API endpoints using Flask/FastAPI
- Integrate backend APIs with the frontend
- Implement GenAI solution and GitHub OAuth
- Document APIs in Swagger/OpenAPI
- Perform initial API testing using Postman

D. Sprint 4: Milestone 5 - Testing & Quality Assurance

Dates: 21/Feb/25 - 07/Mar/25 (2 weeks)

Goals:

- Design comprehensive test cases for backend and frontend
- Implement unit and integration tests using pytest and Selenium
- Conduct automated testing for APIs and UI components
- Perform bug tracking and resolution using Jira

E. Sprint 5: Milestone 6 - Final Prototype & Presentation

Dates: 08/Mar/25 - 30/Mar/25 (3 weeks)

Goals:

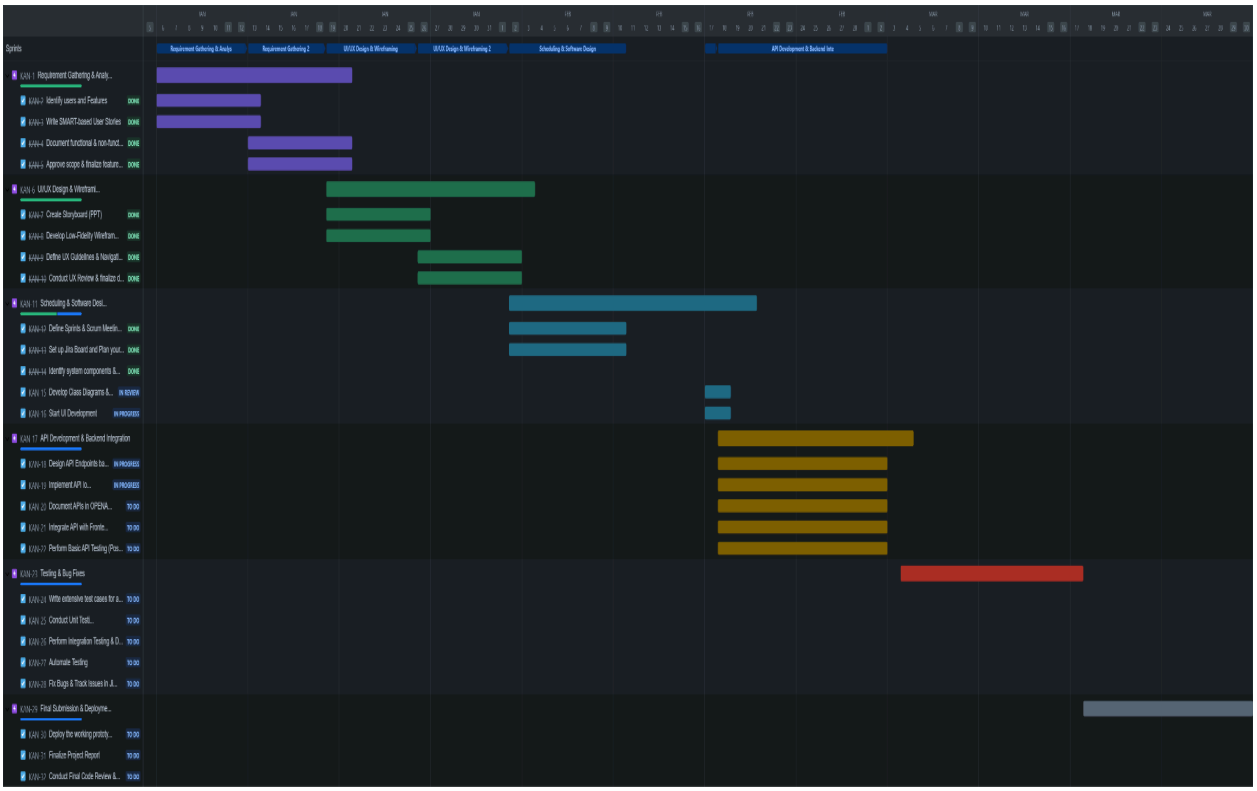
- Prepare the final prototype with all integrations
- Conduct final UI and UX enhancements
- Finalize documentation for the project
- Create and record the final presentation
- Submit the completed project along with code and reports

1.3 Gantt Charts

The Gantt charts provide a visual representation of the project timeline, showing the progression of different epics and sprints along with the tasks involved in each epic.

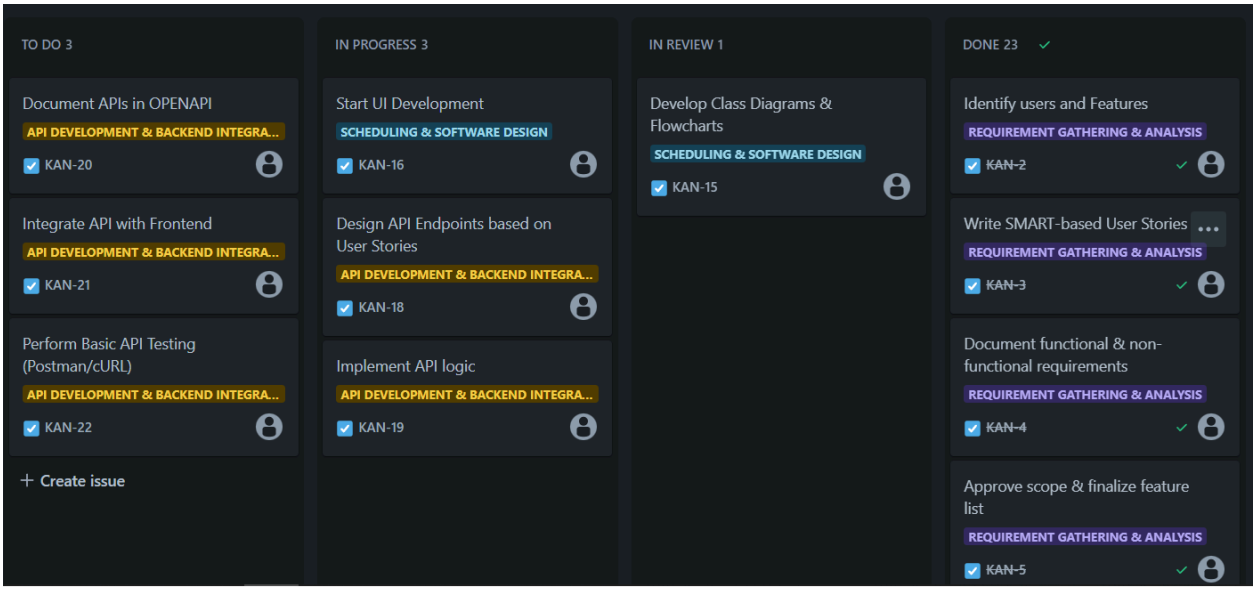
Monthly Timeline View

This view highlights the detailed schedule for each sprint, showing the tasks to be completed week by week. It helps track the progress and deadlines within each sprint for effective task management.



1.4 Scrum Board (Kanban View)

The following section presents a screenshot of the Kanban board used for Sprint 3, providing a clear visualization of tasks assigned to different team members. Each task is categorized based on its current state in the workflow, ensuring efficient task management and real-time progress tracking. The board is divided into columns representing different stages of the work process.



2. Scrum Details

2.1 Scrum Standup Schedule

Scrum meetings or standups are scheduled regularly to discuss the progress on ongoing tasks, assign new tasks, and incorporate any new requirements in the agile methodology. The scrum meetings for our team are scheduled as follows:

- **Every Monday (Client/TA Meet + Team Standup)**

  Duration: 20 minutes + 20 minutes

- **Every Thursday (Team Standup)**

  Duration: 20 minutes

- **Every Saturday (Team Standup)**

  Duration: 20 minutes

2.2 SCRUM Meetings Schedule and Minutes

Below are the details of some of the scrum meetings, including the agenda, minutes, and attendee list for each session.

A. Minutes from Scrum 1 Meeting

(Date: 8th January, Duration: 1 hour, Start time: 8:00pm, End time: 9:00pm)

- **Agenda:**

1. Review and finalization of user requirements for Milestone 1 & Milestone 2.
2. Discussion on UI/UX design principles and wireframes.

- **Description of the Meet:**

  The team discussed the user requirements and worked on finalizing the requirements for the first two milestones. Feedback was provided for the wireframes and design principles. The group also reviewed the primary features for Milestone 1 & 2 and agreed on the tasks to be completed.

- **Attendees:**

1. Shreya
2. Shruti
3. Satyaki
4. Aaruni
5. Adarsh
6. Puja
7. Himanshu

B. Minutes from Client Meeting

(Duration: 30 minutes, Start time: 8:15pm, End time: 8:45pm)

- **Agenda:**

1. Introduction of the project team and their roles.
2. Discussion of feature requirements and design reviews with the client.

- **Description of the Meet:**

  Each team member introduced their role in the project, followed by a discussion of the features and design elements for the project. The client provided feedback on the UI/UX design and confirmed the requirements for the initial milestones. Any concerns and questions were addressed during the meeting.

- **Attendees:**

1. Shreya
2. Shruti
3. Satyaki
4. Aaruni
5. Adarsh
6. Puja
7. Himanshu

C. Minutes from Scrum 3 Meeting

(Date: 1st February, Duration: 1 hour, Start time: 8:00pm, End time: 9:00pm)

- **Agenda:**

1. Review and finalization of the deliverables for Milestone 2.
2. Task distribution for UI components and wireframe implementation.
3. Discussion on the review process and feedback integration.

- **Description of the Meet:**

The team reviewed the deliverables for Milestone 2, including the low-fidelity wireframes and UI components. Feedback from previous meetings was integrated, and any remaining issues with the designs were addressed. Tasks for the creation of UI pages and the implementation of the wireframes were redistributed. The team discussed potential challenges with the current design and scheduled follow-up tasks for the next sprint to ensure smooth progression.

The project schedule was updated to reflect the completion of Milestone 2 deliverables, and the team outlined the steps for Milestone 3. The upcoming deadlines were emphasized, and the need for early reviews was discussed to avoid delays in future milestones.

- **Attendees:**

1. Shreya
2. Shruti
3. Satyaki
4. Aaruni

D. Minutes from Scrum 4 Meeting

(Date: 10th February, Duration: 1 hour, Start time: 8:00pm, End time: 9:00pm)

- **Agenda:**

1. Review of the frontend development progress.
2. Task distribution for frontend pages and API integration.
3. Discuss updates to the project schedule and backlog.

- **Description of the Meet:**

The team discussed the progress of frontend development, identified any blockers, and redistributed tasks to accelerate progress. Tasks for frontend page creation and API

integration were assigned to team members. The project schedule was updated to reflect the current status, and any additional requirements were noted for the next sprint.

- **Attendees:**

1. Shreya
2. Adarsh
3. Puja
4. Himanshu

---

3. Software Component Design

3.1 Overview

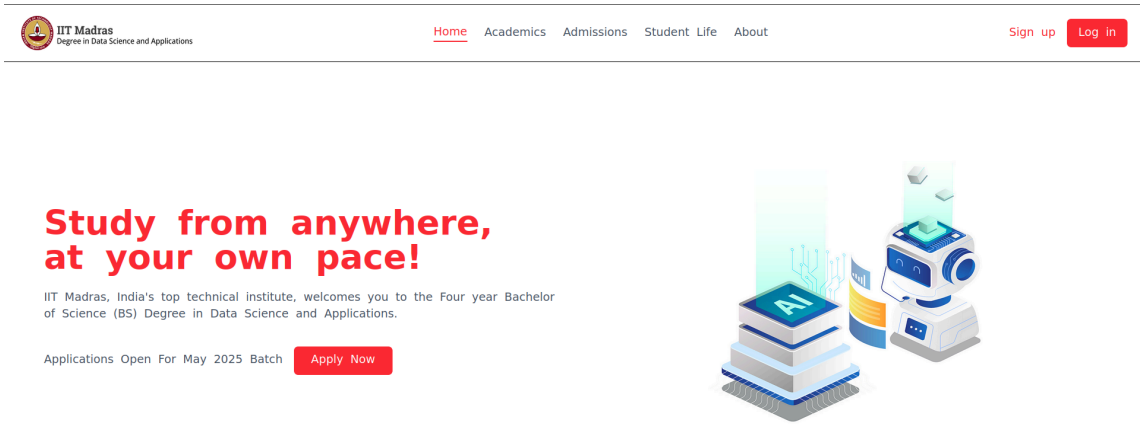
The frontend is a React-based web application that provides authenticated users with a structured learning experience. It includes a courses page with lectures, supplementary resources, CGPA tracking, a course-specific page with various learning tools, and a dashboard visualising user performance.

3.2 Component Breakdown

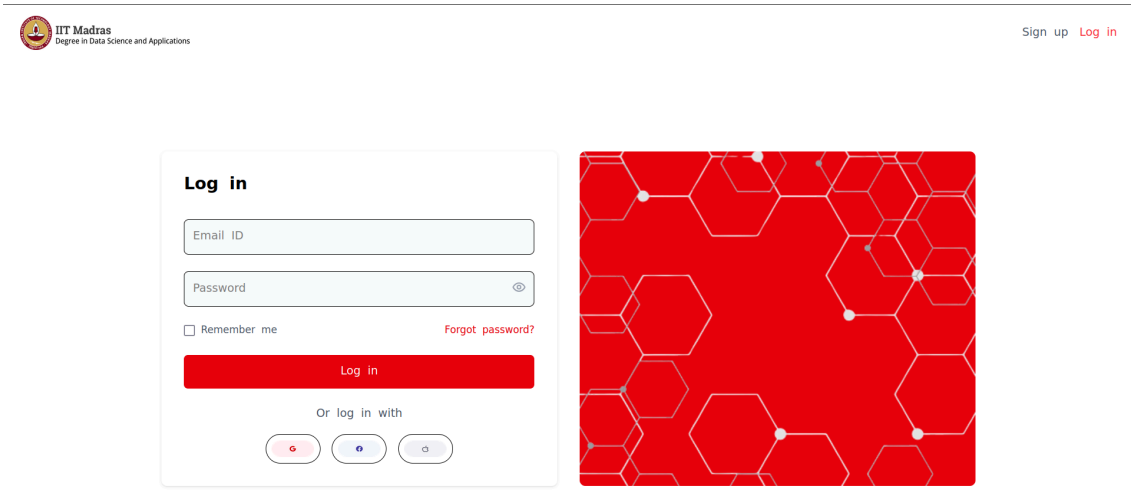
A. Authentication System

- **Purpose:** Manages user authentication and session.
- **Components:**
    - **Login.jsx:** Login card for user authentication.
    - **Signup.jsx:** Signup card for user signup.
- **Inputs:** User credentials (email, password).
- **Processing:** Sends login/signup

- nop request to Flask backend.
- **Outputs:** Stores session data and grants access.



Picture 1 - Landing Page



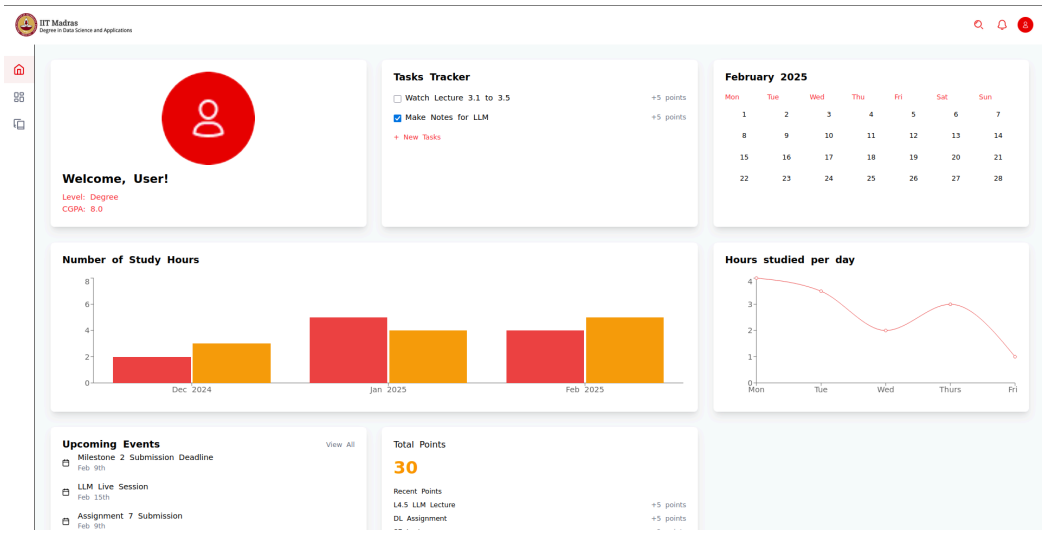
Picture 2 - Login Page

B. Dashboard (Dashboard.jsx)

- **Purpose:** Displays user analytics visually.
- **Features:**
    - Performance charts (marks, time spent, activity trends).

    - Personalized learning insights.

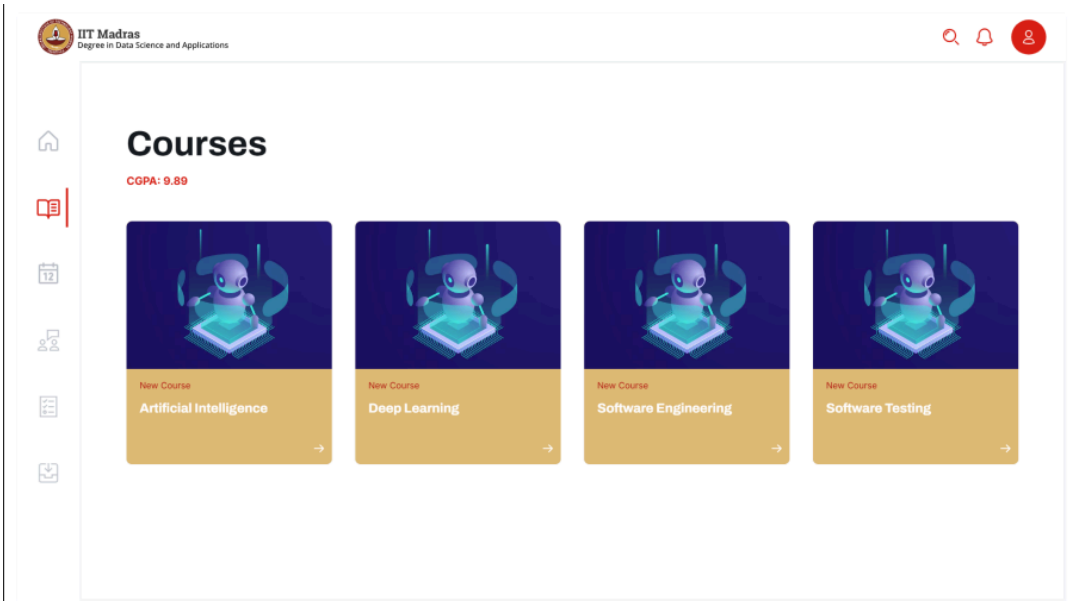
- **Inputs:** User performance data from backend.
- **Processing:** Fetches and processes user-specific analytics.
- **Outputs:** Graphs and visual reports.



Picture 3 - Dashboard Page

C. Courses Page (Courses.jsx)

- **Purpose:** Lists all enrolled courses, links to access their videos and resources, and displays CGPA.
- **Features:**
    - Course cards with progress indicators.
    - CGPA calculation and display.
- **Inputs:** List of courses, grades, progress data.
- **Processing:** Fetches course data and calculates CGPA.
- **Outputs:** Display of enrolled courses and CGPA.



Picture 4 - Courses Page

D. Course-Specific Page (CourseDetail.jsx)

- **Purpose:** Provides access to course materials and tools.
- **Features:**
    - **Lectures Section:** Lists and plays lecture videos.
    - **Assignments Section(Assignment.jsx)**: Links to course assignments.
    - **PYQs Section(PYQs.jsx)**: Provides access to past year questions.
    - **Notes Section(CourseNotes.jsx)**: Allows users to take and save notes.
    - **Chatbot(CourseChatBot.jsx)**: AI-powered chatbot for doubt resolution.
    - **Chat Room(ChatRoom.jsx)**: Real-time chat for course discussions.
- **Inputs:** Course ID, user progress.
- **Processing:** Fetches lectures, assignments, notes, and chat data.
- **Outputs:** Dynamic rendering of course-specific content.

Picture 5 - Chatroom Page

Picture 6 - Chatbot Page

3.3 Backend Component (Flask)

The backend will be built using Flask-RESTful to create a RESTful API responsible for communication between the frontend and database while serving as the controller layer in the MVC architecture. The backend will use Flask-SQLAlchemy as the ORM and integrate other necessary Python packages based on project requirements.

A. User Management

- Handles user authentication, registration, and role-based access control (**Admin, Instructor, Student, and TA**).
- Implements **JWT-based authentication** for secure API access.
- Provides CRUD APIs for managing user accounts and permissions.

B. Course and Enrollment Management

- Manages **courses, lectures, assignments, and student enrollments**.
- Provides APIs for CRUD operations on courses and their contents.
- Role-based access control:
    - **Instructors** can create, update, and manage courses and assignments.
    - **Students** can enroll, access materials, and submit assignments.
    - **TAs** assist with grading and student queries.

C. AI Assistant and Guidance System

- Uses **Retrieval-Augmented Generation (RAG)** to fetch relevant content from course materials.
- Enforces **academic integrity** by restricting direct solutions for assessments.

D. Document Analysis

- Interacts with **Generative AI** APIs to analyze student-submitted documents.
- Extracts summaries, highlights important points, and provides **constructive feedback**.
- Detects plagiarism and improper citations using NLP techniques.

E. Notification & Background Jobs

- **Celery** workers handle batch jobs and async tasks such as:

    - Scheduled **reminders** for deadlines and events.
    - **Auto-grading** of certain assignments.
    - **Data processing** for analytics.
- **Celery Beat** is used for scheduling tasks at predefined intervals.
- **Redis** serves as a cache layer and **message broker**, while **RabbitMQ** can be used for handling event-driven notifications.

3.4 Business Logic and Data Processing

The middleware layer ensures seamless operations and data integrity by handling:

- **Course & Assignment Management:** Manages course creation, assignments, deadlines, and grading.
- **Student Analytics:** Tracks progress, generates performance insights, and provides study recommendations.
- **AI Study Assistance:** Uses NLP to answer queries while ensuring academic integrity.
- **GitHub Integration:** Fetches commit history, tracks coding progress, and analyzes contributions.
- **Team Management:** Handles team creation, role assignments, and collaboration tools.

This layer validates data, performs CRUD operations, and maintains consistency across all processes.

3.5 Security and Authentication

- **Authentication:** Verifies user credentials for students, instructors, TAs, and admins.
- **Role-based Authorization:** Enforces access control based on user roles (admin, instructor, TA, student).

- **Data Privacy:** Ensures course materials, assignments, and student data are accessible only to authorized users.

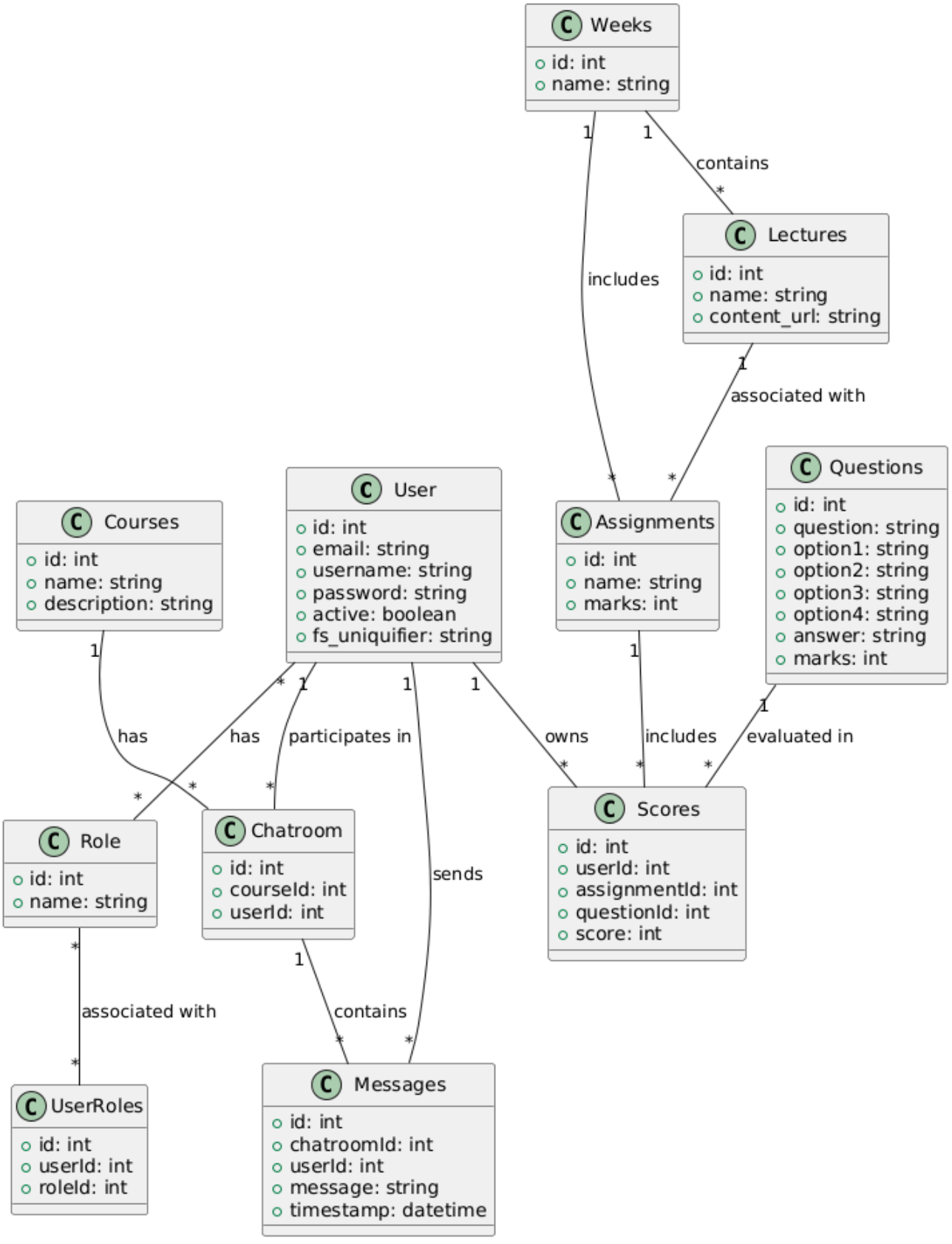
3.6 Third-Party Integrations

- **Google OAuth:**
    - Enables secure authentication and user login.
    - Provides role-based access control for students, instructors, and admins.
- **Generative AI API:**
    - Analyzes submitted assignments and provides insights.
    - Generates feedback on documentation quality and improvements.
- **Email/SMS API:**
    - Sends automated reminders for deadlines and tasks.
    - Delivers performance reports to relevant stakeholders.

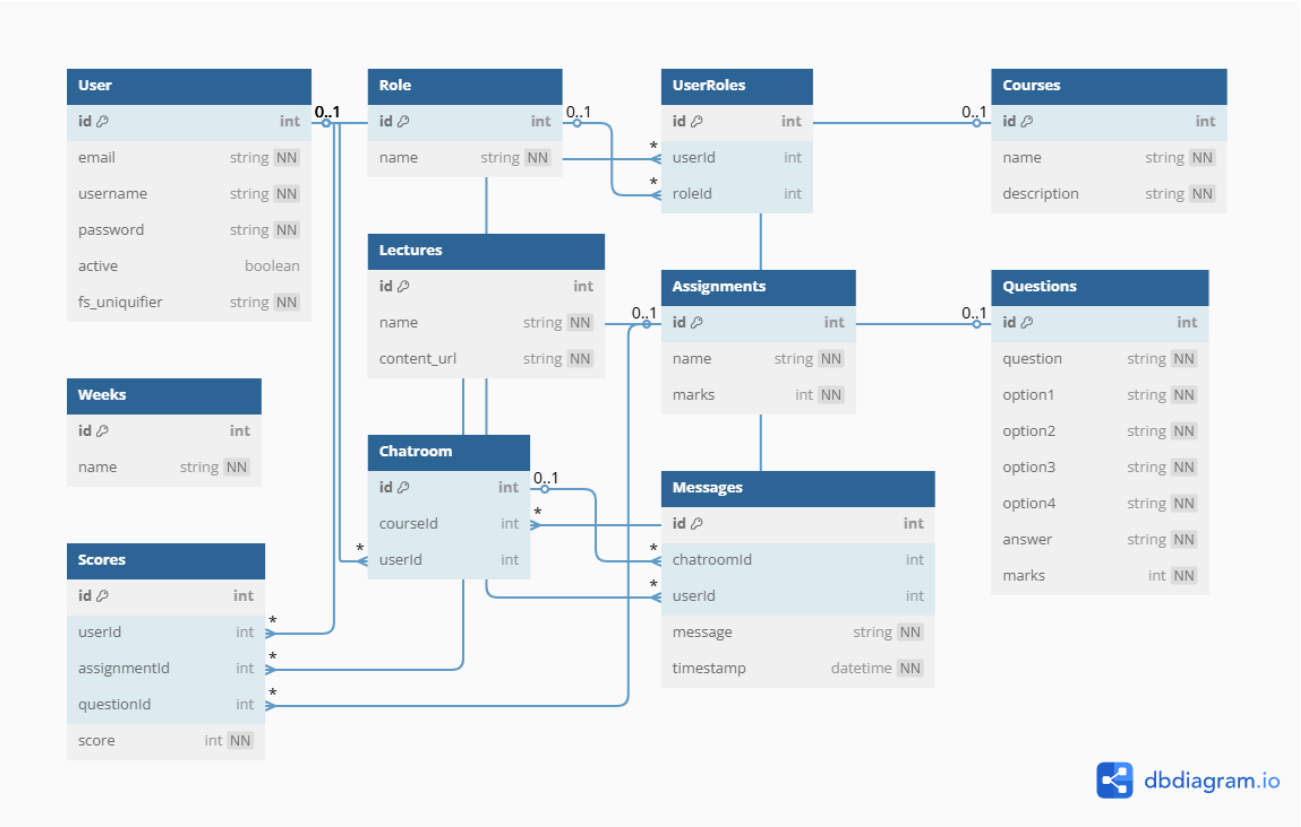
3.7 Database Layer

- **Centralized storage for:**
    - Users, courses, modules, assignments, discussions, and AI-generated insights.
- **Relational Database:**
    - Uses SQLite for development due to its simplicity and efficiency.
    - In production, PostgreSQL or MySQL will be used for scalability and performance.
- **Document Storage:**
    - If required, MongoDB can be used for storing unstructured data such as student submissions and AI-processed documents.

4. Class Diagram



5. ER Diagram


User
id int
email string NN
username string NN
password string NN
active boolean
fs_uniquifier string NN
Role
id int
name string NN
UserRoles
id int
userId int
roleId int
Courses
id int
name string NN
description string NN
Lectures
id int
name string NN
content_url string NN
Assignments
id int
name string NN
marks int NN
Questions
id int
question string NN
option1 string NN
option2 string NN
option3 string NN
option4 string NN
answer string NN
marks int NN
Weeks
id int
name string NN
Chatroom
id int
courseId int
userId int
Messages
id int
chatroomId int
userId int
message string NN
timestamp datetime NN
Scores
id int
userId int
assignmentId int
questionId int
score int NN
0..1
*
dbdiagram.io